Reinforcement Learning - Concepts, Methods and Appeal to Economics

Inspired by Iskhakov, Rust, Schjerning (2020), "Machine Learning and Structural Econometrics: Contrast and Synergies"

November 30, 2022

GEORGETOWN UNIVERSITY

Introduction

Introduction

- Decision-making problems involve finding the optimal policy in a complex environment. Brute-force optimization is typically infeasible.
- If the environment can be broken into **repeated** and **identical** series of smaller problems, we call it a **Markovian Decision Processes (MDPs)**.
- MDPs have special solution methods that exploit this identical and repeated structure.

Introduction

- Decision-making problems involve finding the optimal policy in a complex environment. Brute-force optimization is typically infeasible.
- If the environment can be broken into **repeated** and **identical** series of smaller problems, we call it a **Markovian Decision Processes (MDPs)**.
- MDPs have special solution methods that exploit this identical and repeated structure.

- Examples
  - Compsci: Shortest Paths in a Graph, String algorithms, Viterbi Algorithm
  - Finance: Consumption-Saving, Optimal Portfolio, Trade Execution.
  - Robotics: Self-Driving Cars, Robots, Chess Engines, Chat-bots.
  - Policy: Social Security, Public good replacement
  - Business: Inventory management, recommender system, A/B testing

Introduction

- Decision-making problems involve finding the optimal policy in a complex environment. Brute-force optimization is typically infeasible.
- If the environment can be broken into **repeated** and **identical** series of smaller problems, we call it a **Markovian Decision Processes (MDPs)**.
- MDPs have special solution methods that exploit this identical and repeated structure.
- Examples
  - Compsci: Shortest Paths in a Graph, String algorithms, Viterbi Algorithm
  - Finance: Consumption-Saving, Optimal Portfolio, Trade Execution.
  - Robotics: Self-Driving Cars, Robots, Chess Engines, Chat-bots.
  - Policy: Social Security, Public good replacement
  - Business: Inventory management, recommender system, A/B testing
- **Reinforcement Learning (RL)** focuses on some key computational aspects of decision and uncertainty in the context of MDPs

Contents

Markov Decision Process

Markov Reward Process

Markov Reward Process (S, P, R, γ)

- (S, P) is a Markov Process, with state space S and transition matrix P
- R is a $|\ S\ |$ dimension reward vector that measures the reward at being in any state.
- γ is the discount factor

Markov Reward Process

Markov Reward Process (S, P, R, γ)

- (S, P) is a Markov Process, with state space S and transition matrix P
- R is a $\mid S \mid$ dimension reward vector that measures the reward at being in any state.
- γ is the discount factor

- Given initial state S_0, H^i is the ith full history of (state,reward) realizations.
- Return G^i is a discounted sum of rewards in H^i.
- Value $V(s)$ is the expected return of starting at $S_0 = s$.

Markov Reward Process

Markov Reward Process (S, P, R, γ)

- (S, P) is a Markov Process, with state space S and transition matrix P
- R is a $\mid S \mid$ dimension reward vector that measures the reward at being in any state.
- γ is the discount factor

- Given initial state S_0, H^i is the ith full history of (state,reward) realizations.
- Return G^i is a discounted sum of rewards in H^i.
- Value $V(s)$ is the expected return of starting at $S_0 = s$.

EXAMPLE

- $S = [0, 1], P = \begin{bmatrix} 0.3 & 0.7 \\ 0.1 & 0.9 \end{bmatrix}, R = [-1, +1], \gamma = 0.9, S_0 = 0$
- $H^0 = \{(0, -1), (1, +1), (1, +1), (1, +1)...\}, G^0 = 20.5$
- $H^1 = \{(0, -1), (0, -1), (1, +1), (1, +1)...\}, G^1 = 18.5$
- $V(0) = E[G^i | S_0 = 0] = 20$

Markov Decision Process and Policies

Markov Decision Processes

A MDP is (S, P, R, γ and A), with:

- A is a set of possible actions an agent can take.
  - Actions influence the state evolution:
    - For every action $a \in A$, P^a is a $|S|$x$|S|$ transition matrix.
    - $P^a_{s,s'} = Prob(S_{t+1} = s' | S_t = s, A_t = a)$
  - Actions influence the rewards received:
    - R^a_s is reward for taking action a in state s.

Markov Decision Process and Policies

Markov Decision Processes

A MDP is (S, P, R, γ and A), with:

- A is a set of possible actions an agent can take.
  - Actions influence the state evolution:
    - For every action $a \in A$, P^a is a $|S|$x$|S|$ transition matrix.
    - $P^a_{s,s'} = Prob(S_{t+1} = s' | S_t = s, A_t = a)$
  - Actions influence the rewards received:
    - R^a_s is reward for taking action a in state s.

Policies

A policy π is a comprehensive strategy regarding actions.

- π is time-invariant, state-dependent.
- π is a |A|x|S| matrix s.t. $\pi_{a,s} = Prob(A_t = a | S_t = s)$
- π covers both stochastic/experimental and deterministic/certain decision-making.

Optimal Policy

Against any policy π, a MDP (S, P, R, γ and A) gives rise to a Markov Reward Process $(S, P^\pi, R^\pi, \gamma)$, with

- $P^\pi_{s,s'} = \sum_a \pi_{a,s} P^a_{s,s'}$
- $R^\pi_s = \sum_a \pi_{a,s} R^a_s$

Value function for policy π

Consider π a policy on S for some infinite-horizon problem.

To π we can associate value function V^π:

$$V^\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi(s_t)) \mid s_0 = s; \pi\right]$$

where $s_{t+1} \sim p(\cdot \mid s_{t-1}, a_t = \pi(s_t)) \qquad \forall t \in \mathbb{N}$

Can also introduce Bellman operator $\mathcal{T}^\pi$, to which V^π is the only fixed point...

Optimal Policy

Optimal policy and value function

The **optimal policy** brings the maximum expected return from a MDP:

- Optimal policy: $\pi^*(s) \in \text{argmax}_{\pi \in \Pi} V^\pi$
- Optimal value fct: $V^*(s) = \max_{\pi \in \Pi} V^\pi(s) = V^{\pi^*}(s)$

Optimal Policy

Optimal policy and value function

The **optimal policy** brings the maximum expected return from a MDP:

- Optimal policy: $\pi^*(s) \in \text{argmax}_{\pi \in \Pi} V^\pi$
- Optimal value fct: $V^*(s) = \max_{\pi \in \Pi} V^\pi(s) = V^{\pi^*}(s)$

Action Value function Q

$Q^*(a, s)$ is the value of choosing action a in state s; given optimal policy will be followed in following periods. Q^π is defined analogously.

Optimal Policy

Optimal policy and value function

The **optimal policy** brings the maximum expected return from a MDP:

- Optimal policy: $\pi^*(s) \in \text{argmax}_{\pi \in \Pi} V^\pi$
- Optimal value fct: $V^*(s) = \max_{\pi \in \Pi} V^\pi(s) = V^{\pi^*}(s)$

Action Value function Q

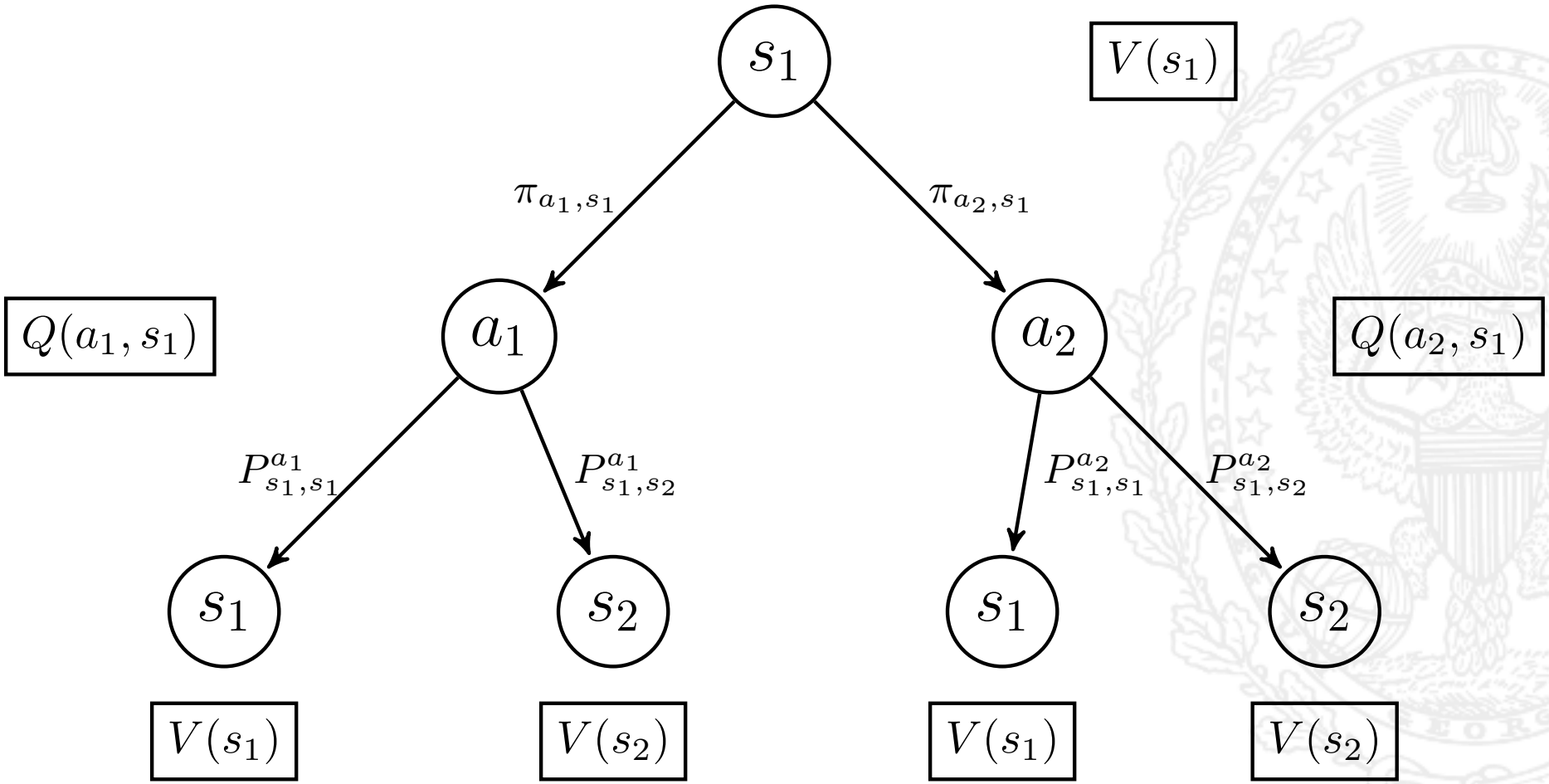
$Q^*(a, s)$ is the value of choosing action a in state s; given optimal policy will be followed in following periods. Q^π is defined analogously.

Bellman Equations

We can write V and Q recursively as:

- $Q(a, s) = R_s^a + \gamma \sum_{s'} P_{s,s'}^a max_{a'} Q(a', s')$
- $V(s) = \max_a (R_s^a + \sum_{s'} P_{s,s'}^a V(s'))$

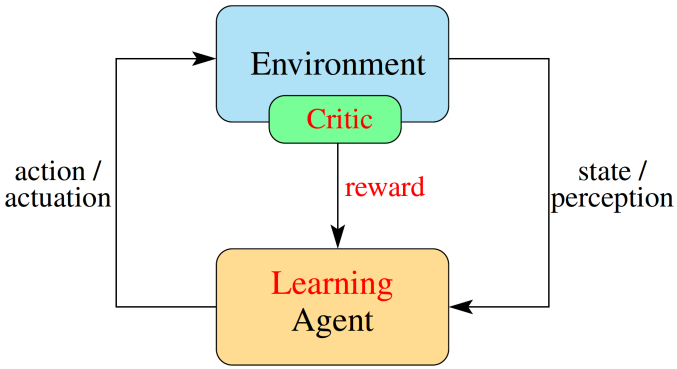
State-Value and Action-Value: example



- If π, V, Q are optimal:
  - $V(s) = \sum_a \pi_{a,s} Q(a,s) = \max_{a'} Q(a',s)$
  - $Q(a,s) = R^a_s + \gamma \sum_{s'} P^a_{s,s'} V(s')$

Reinforcement Learning

Reinforcement Learning



Conceptual environment of reinforcement learning

For $i = 1, \ldots, n$

1. Set $t = 0$
2. Set initial state x_0 *[possibly random]*

   [execute one trajectory]
3. **While** (x_t not terminal)
   - 3.1 Take action a_t
   - 3.2 Observe next state x_{t+1} and reward r_t
   - 3.3 Set $t = t + 1$

   EndWhile

EndFor

Reinforcement Learning - Setting and Objective

Setting

No information about R, P but we have access to **generative model** f s.t.$(r, x') = f(x, a)$

- Episodic - at state x with policy π, we can simulate forward trajectories $((x_1^i = x, a_1^i, r_1^i), (x_2^i, a_2^i, r_1^i)...(x_{T^i}^i, a_{T^i}^i, r_{T^i}^i))$.
- Online - at time t, at state x_t, taking action a_t, agent observes only r_t, x_{t+1}.

Reinforcement Learning - Setting and Objective

Setting

No information about R, P but we have access to **generative model** f s.t.$(r, x') = f(x, a)$

- Episodic - at state x with policy π, we can simulate forward trajectories $((x_1^i = x, a_1^i, r_1^i), (x_2^i, a_2^i, r_1^i)...(x_{T^i}^i, a_{T^i}^i, r_{T^i}^i))$.
- Online - at time t, at state x_t, taking action a_t, agent observes only r_t, x_{t+1}.

How to find optimal policy?

- Two families of methods: **value function iteration** and **policy iteration**
- Two crucial steps used by both methods:
  - Policy Evaluation: Using π, evaluate V^π and/or Q^π.
  - Policy Improvement: Using $\hat{V}^\pi$ or Q^π, find a better/best policy π'.
- Either exact or approximate dynamic programming

VFI & policy iteration: Comparative analysis

Value Iteration

1. Guess: $V_0(s)$
2. $\forall k \in [1..K]$, update:

$$V_{k+1}(s) = \max_a (R_s^a + \sum_{s'} P_{s,s'}^a V_k(s'))$$

3. Compute: $\pi(s) = \text{argmax}_a (R_s^a + \sum_{s'} P_{s,s'}^a V_K(s'))$

Policy Iteration

1. Guess π_0
2. $\forall k \in [1..K]$ until convergence:

   Evaluate: Estimate V^{π_k} via:
   - Monte-Carlo
   - Temporal Difference learning
   - iterating $\mathcal{T}^\pi$

   Improve policy
   - Greedily: $\pi_{k+1}(x) \in \text{argmax}_a [r(x,a) + \gamma \mathbb{E}[V^{\pi_k}(y)|x,a]]$
   - Fancily: **SARSA** introduces exploration-exploitation trade-off. **Q-learning** applies temporal difference to function Q

Policy Evaluation

We are given some policy π from which we wish to estimate V^π.

Monte Carlo - "Bag of trajectories"

- At arbitrary state x_0, generate N trajectories.
- Let t_k^i the index of first observation of x_k in ith trajectory.
- $\forall x_k$, compute $\hat{V}^\pi(x_k) = (1/N)\sum_{i=1}^{N}\sum_{t=t_k^i}^{T_i}\gamma^{t-t_k^i}r_t^i$
- First-visit vs every-visit Monte-Carlo

Temporal Difference - Iterating on trajectories

- TD(1) - Update at end of trajectory:
  - $\hat{V}_i^\pi(x_k) = \boldsymbol{(1-\alpha_i)}\hat{V}_{i-1}^\pi(x_k) + \boldsymbol{\alpha_i}\sum_t \gamma^t r_t^i$
- TD(0) - Smart update at every realization.
  - $\hat{V}_{i,t}^\pi(x_k) = \boldsymbol{(1-\alpha_{i,k})}\hat{V}_{i,t-1}^\pi(x_k) + \boldsymbol{\alpha_{i,k}}\delta_t$
  - Bellman Error: $\delta_t = r_t^i + \hat{V}_{i,t-1}(x_{k+1}) - \hat{V}_{i,t-1}(x_k)$

  $\alpha_{i,k}$ is a state-specific **learning rate**
- TD(λ), $\lambda \in (0,1)$: Eligibility traces

VFI & Policy iteration: Comparative analysis

Value Iteration

- Identifies V^* by iterating the optimal Bellman operator until approximate stability
- γ-contraction of $\mathcal{T}$ provides guaranteed convergence rate. For precision criterion $\epsilon > 0$, the maximum number of iterations necessary is:

$$K = \frac{\log\left(\max_{a \in A} u(a)/\epsilon\right)}{\log\left(1/\gamma\right)}$$

- Stochastic "TD-style" versions exist and backed-out theoretically

Policy Iteration

- repeatedly identified the greedy policy and compute the associated value function
- generates a sequence of policies with uniformly non-decreasing performance
- and converges to π^* in a finite number of steps

VFI & Policy iteration: Comparative analysis

Value Iteration

Identifies V^* by iterating the optimal Bellman operator until approximate stability

Pros: Each iteration is computationally efficient

Cons: Convergence is only asymptotic

Policy Iteration

generates a sequence of policies with uniformly non-decreasing performance, and converges to π^* in a finite number of steps

Pros: Converges in a finite (small) number of iterations

Cons: Each iteration requires a full policy evaluation

Exploration/exploitation trade-off

- Exploring all possible policies and learning about their value is infeasible in practice.

Exploration/exploitation trade-off

- Exploring all possible policies and learning about their value is infeasible in practice.
- Still we want our algorithm to explore some practices that could improve on the current best solution.

Exploration/exploitation trade-off

- Exploring all possible policies and learning about their value is infeasible in practice.
- Still we want our algorithm to explore some practices that could improve on the current best solution.

$$a_t = \text{argmax}_a[Q(X_t, a)] \quad \text{No convergence}$$
$$a_t \sim \mathcal{U}(\mathcal{A}) \quad \text{Poor rewards}$$

Exploration/exploitation trade-off

- Exploring all possible policies and learning about their value is infeasible in practice.
- Still we want our algorithm to explore some practices that could improve on the current best solution.

$$a_t = \text{argmax}_a[Q(X_t, a)] \quad \text{No convergence}$$
$$a_t \sim \mathcal{U}(\mathcal{A}) \quad \text{Poor rewards}$$

This is the **exploration/exploitation trade-off**.

Exploration/exploitation trade-off

- Exploring all possible policies and learning about their value is infeasible in practice.
- Still we want our algorithm to explore some practices that could improve on the current best solution.

$$a_t = \text{argmax}_a[Q(X_t, a)] \quad \text{No convergence}$$
$$a_t \sim \mathcal{U}(\mathcal{A}) \quad \text{Poor rewards}$$

This is the **exploration/exploitation trade-off**.

Stochastic multi-arm bandit

The simplest framework to tackle this trade-off simplifies MDPs down to (A, r):

- A the action space
- $r(a)$ the reward of action a

OBJECTIVE **learn** π^* as **efficiently** as possible

Exploration/exploitation trade-off

The agent has $i = 1, ..., K$ arms to pull

At each round $t = 1, ..., n$

- Simultaneously
  - The environmental chooses a vector of rewards $\{X_{i,t}\}_{i=1}^{K}$
  - The agent chooses an arm I_t
- The learner receives a reward $X_{I_t,t}$
- The environment does not reveal the rewards of the other arms

Regret and the trade-off

Problem 1: environment does not reveal the rewards of the arms not pulled by the learner

$\Longrightarrow$ the learner should gain information by repeatedly pulling all the arms

Exploration

Problem 2: Whenever the learner pulls a bad arm, it suffers some regret

$\Longrightarrow$ the learner should reduce the regret by pulling an optimal arm sufficiently often

Exploitation

Exploration/Exploitation Trade-off!

Regret and the trade-off

Problem 1: environment does not reveal the rewards of the arms not pulled by the learner

$\Longrightarrow$ the learner should gain information by repeatedly pulling all the arms

Exploration

Problem 2: Whenever the learner pulls a bad arm, it suffers some regret

$\Longrightarrow$ the learner should reduce the regret by pulling an optimal arm sufficiently often

Exploitation

Exploration/Exploitation Trade-off!

Quantifying sub-optimality: Regret

The cumulative sub-optimality of decisions is captured by a function called **regret** R_n:

$$R_n(\mathcal{A}) = \max_{i=1,..,K} \mathbb{E}\left[\sum_{t=1}^{n} X_{i,t}\right] - \mathbb{E}\left[\sum_{t=1}^{n} X_{I_t,t}\right]$$

Regret and the trade-off

Problem 1: environment does not reveal the rewards of the arms not pulled by the learner

$\Longrightarrow$ the learner should gain information by repeatedly pulling all the arms

Exploration

Problem 2: Whenever the learner pulls a bad arm, it suffers some regret

$\Longrightarrow$ the learner should reduce the regret by pulling an optimal arm sufficiently often

Exploitation

Exploration/Exploitation Trade-off!

Problem 3: The environment is stochastic, ie $X_{i,t} \sim \nu_i$

$\Longrightarrow$ The algorithm must keep track of an estimate value (and/or bounds) of each arm!

Regret and the trade-off

Sufficient statistics for regret

One can show that in this environment, the regret rewrites as:

$$R_n(\mathcal{A}) = \sum_{i \neq i^*} \mathbb{E}\left[T_{i,n}(\mu_{i^*} - \mu_i)\right]$$

with $T_{i,n}$ the expected number of pulls of the sub-optimal arm i

$\Longrightarrow$ Sufficient statistics: $(T_{i,n}(\mu_{i^*} - \mu_i))_{i \neq i^*}$

Regret and the trade-off

Sufficient statistics for regret

One can show that in this environment, the regret rewrites as:

$$R_n(\mathcal{A}) = \sum_{i \neq i^*} \mathbb{E}\left[T_{i,n}(\mu_{i^*} - \mu_i)\right]$$

with $T_{i,n}$ the expected number of pulls of the sub-optimal arm i

$\Longrightarrow$ Sufficient statistics: $(T_{i,n}(\mu_{i^*} - \mu_i))_{i \neq i^*}$

This motivates an interesting approach: optimism in the face of uncertainty!

Applied Example: AlphaGo Engine

Deep RL & Representation learning

Deep Reinforcement Learning

- Many real-world decision problems are in very high-dimension (very large number of states of the MDP):
  - stream of images from a camera
  - positions and sensors stream of a robot
- The idea behind **deep RL** is to use neural networks to learn either π, V, Q or some other function used in learning

Representation learning

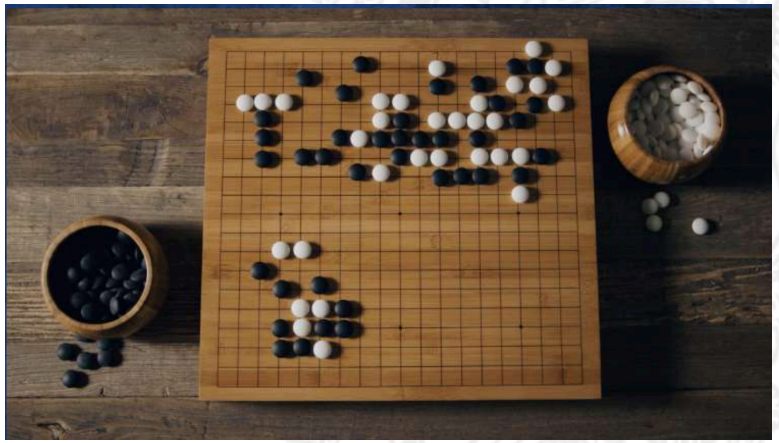
Representation learning aims to learn a representation of the state space and action space on which those functions are more regular

- Intuition: some representations of the state space are more suitable to straightforward learning
- Solutions from ML: **Autoencoders** and Variational AEs: unfolding manifolds and parsimonious latent representations of data

AlphaGo - Context

Go is an adversarial strategy board game for two players.

- Goal: surround more territory than your opponent.
- The players take turns placing the stones (the playing pieces) on the vacant intersections of a square board (19x19).
- When a stone is surrounded by enemy stones, it is captured.



The game proceeds until neither player wishes to make another move. When a game concludes, the winner is determined by counting each player's surrounded territory, captured stones and other bonuses.

Despite very simple rules, Go is **combinatorially very complex**.

AlphaGo - Context

Go is considered impossible to brute-force, mainly for two reasons:

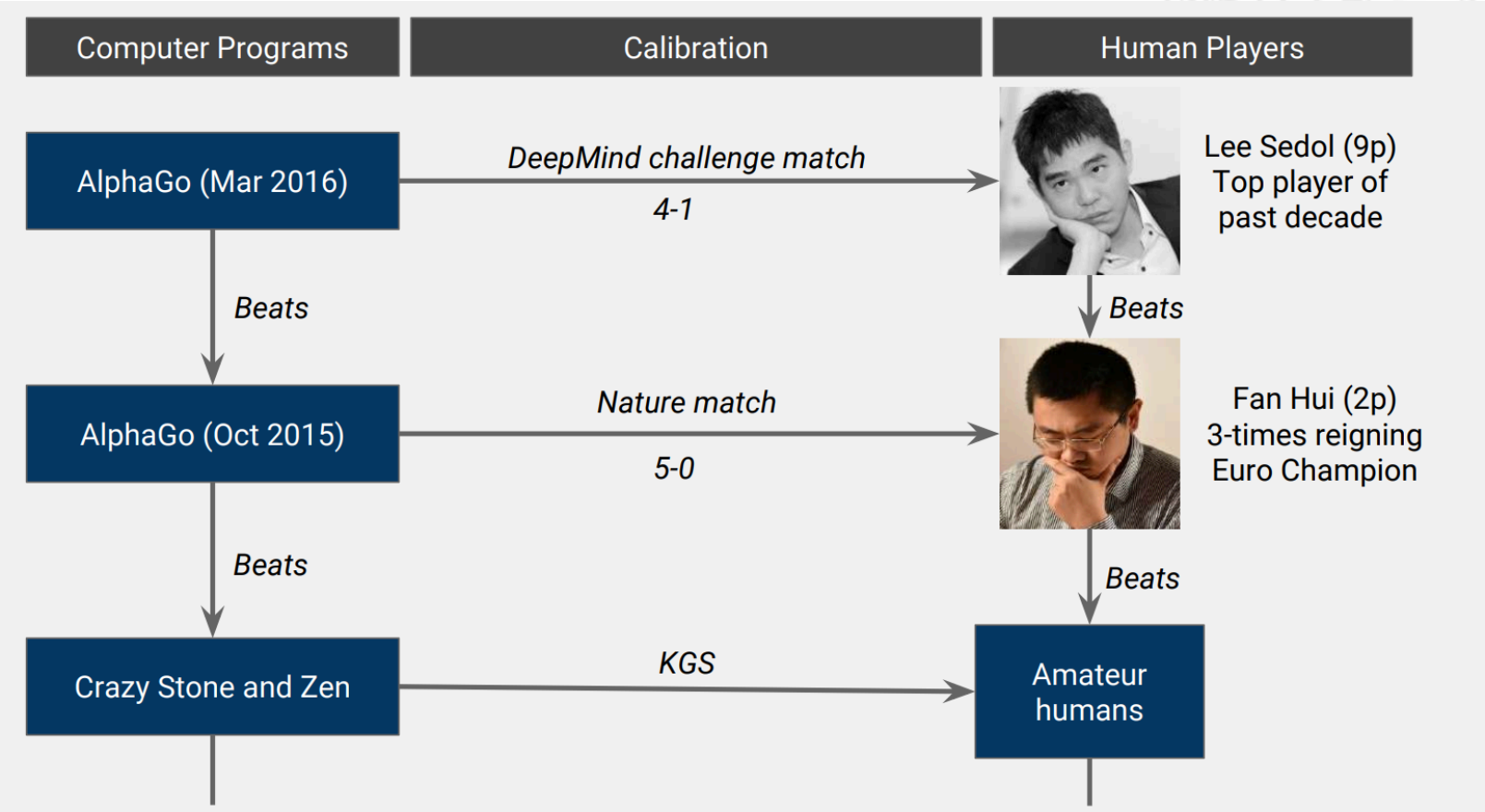
- Go is a game with a very high-dimensional state space: the game tree has complexity 3^{d^2}, where d is the dimension of the board.

  A full board has $d = 19$, implying about $1.74 \cdot 10^{172}$ states

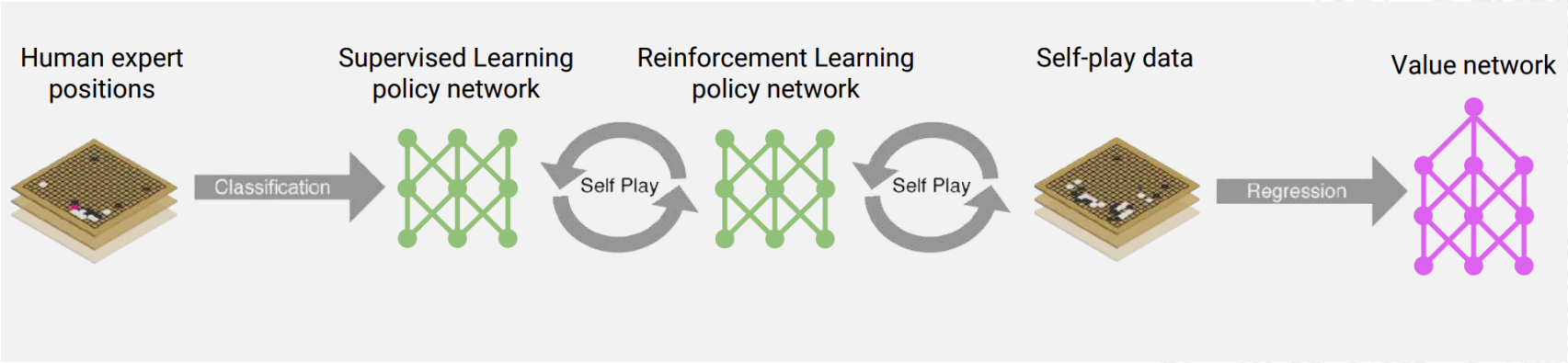
- The reward process is extremely sparse: a binary win/lose at the end of a long game. At most stages of the game, it is really hard to evaluate who is effectively winning.

AlphaGo - History & Significance

In 2015, AlphaGo (DeepMind), was the first algorithm to beat a human professional Go player. In 2016, it beat the world champion.



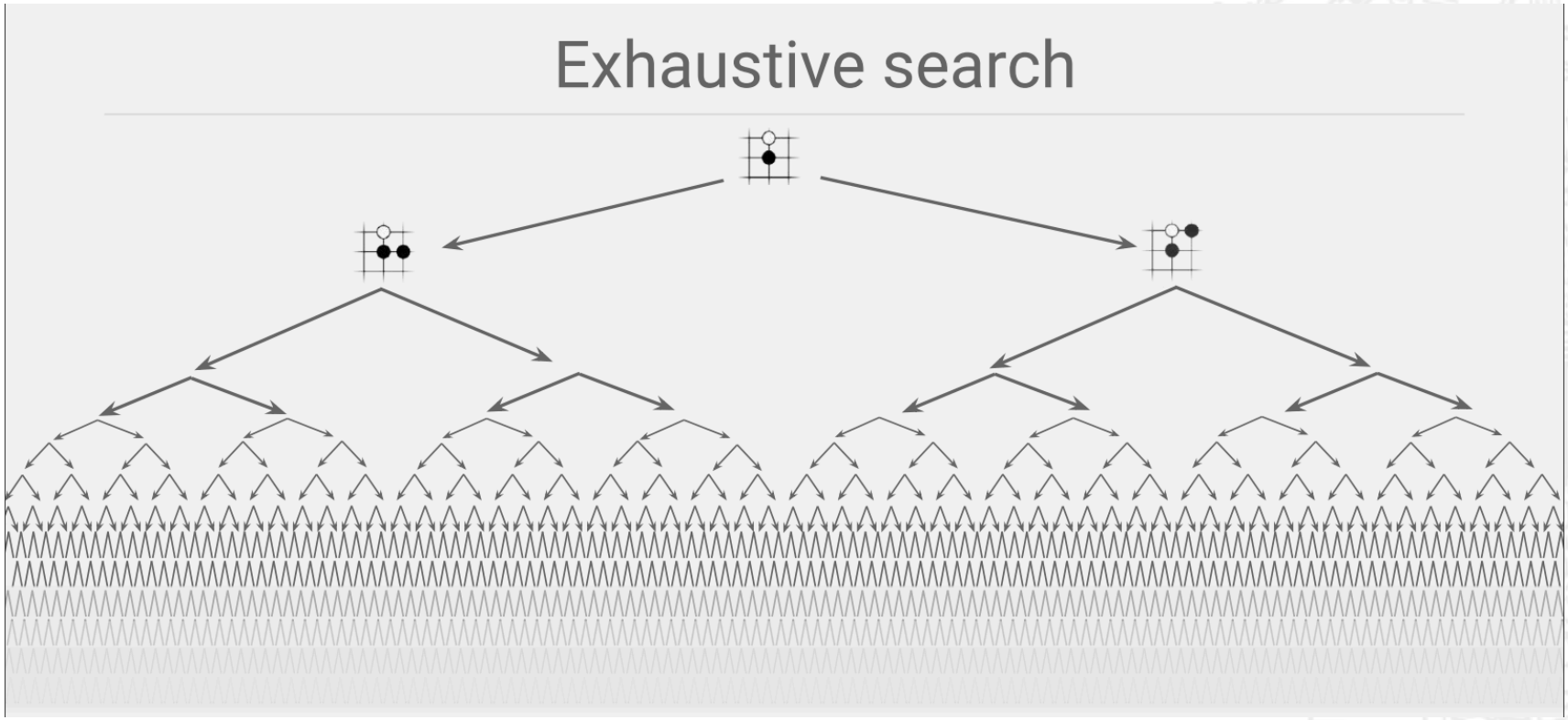
AlphaGo - Principle



- Current policy and value function are both learned with (separate) neural networks
- After a supervised learning phase based on analysing the outcome of past master-level games, the algorithm improves on its policy by playing against himself
- The data from self-play is then used to regress a value function using a NN

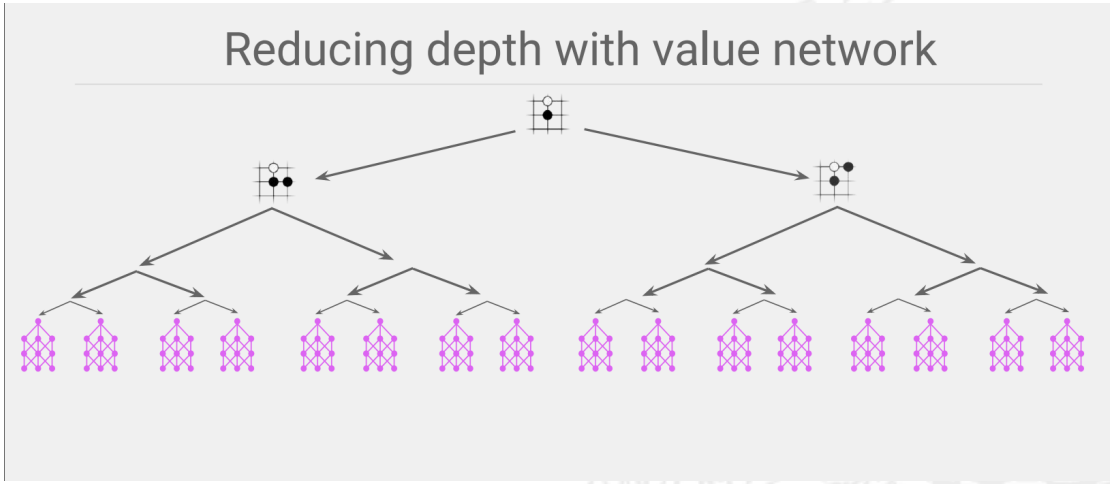
AlphaGo - The original curse of dimensionality

An exhaustive search for an optimal policy yields the curse of dimensionality

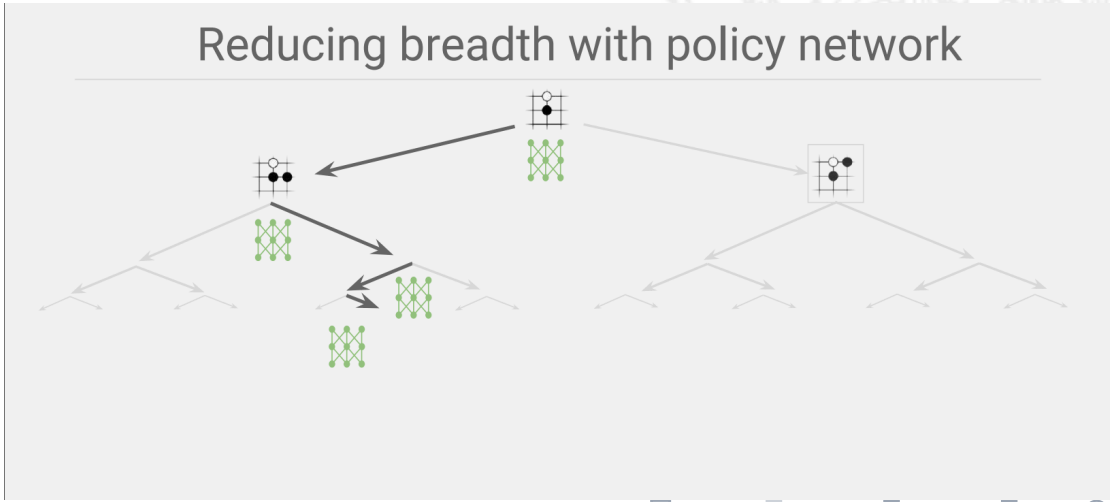


AlphaGo's solution to the CoD problem

Approximate V^* with a value neural net



Approximate π^* with a policy neural net



Synergies with Structural Economics

Objective and Methods

Structural Economics

- Seek to recover policy-invariant parameters (preferences, technology) and conduct welfare analysis.
- Data assumed to be generated by optimizing agents with model-consistent expectations in markets that clear.
- Reward function is represented by utility/profit functions, and model's solution (at steady-state) define the evolution of the data.
- Model is made to fit the data either by MLE or by calibrating parameter value to fit long-run target moments.

Objective and Methods

Structural Economics

- Seek to recover policy-invariant parameters (preferences, technology) and conduct welfare analysis.
- Data assumed to be generated by optimizing agents with model-consistent expectations in markets that clear.
- Reward function is represented by utility/profit functions, and model's solution (at steady-state) define the evolution of the data.
- Model is made to fit the data either by MLE or by calibrating parameter value to fit long-run target moments.

Reinforcement Learning

- Seeks to find the optimal policy in an uncertain environment with fixed reward and transition parameters.
- Agents can be allowed to know the reward/transition matrices or only the reward/transition in a given state.

Areas of Cross-Fertilization

Structural Economics

- RL can be used to model a variety of departures from perfect rationality.
- Multi-agent RL can be used to model strategic behavior (e.g. Oligopoly)
- RL can be used to solve the model when the model transition and reward functions cannot be precisely written down.
- Deep learning can be used to approximate highly nonlinear functions.

Areas of Cross-Fertilization

Structural Economics

- RL can be used to model a variety of departures from perfect rationality.
- Multi-agent RL can be used to model strategic behavior (e.g. Oligopoly)
- RL can be used to solve the model when the model transition and reward functions cannot be precisely written down.
- Deep learning can be used to approximate highly nonlinear functions.

Reinforcement Learning

- SE throws light on the identification problem - where multiple reward functions can lead to identical behavior.
- SE provides a focus on "policy-invariant" parameters which would also be of interest to marketing, business, and finance.
- SE Game theoretic solutions can be used to provide and test benchmarks for multi-agent RL.